tagged, which may not be an acceptable output.

```
>>> nltk.pos_tag(b)
[('hi', 'NN'), ('sharon', 'NN'), ('.', '.'), ('how', 'WRB'),
('are', 'VBP'), ('you', 'PRP'), ('?', '.'), ('revert', 'VB'),
('me', 'PRP'), ('to', 'TO'), ('sharon', 'VB'), ('@', 'NNP'),
('gmail.com', 'NN')]
```

Here, NN stands for noun, WRB for wh-adverb and VBP for present tense verb. Explaining the meaning of all POS tags is beyond the scope of this article. The following command will give details about all the Pen TreeBank POS tags:

```
>>> nltk.help.upenn_tagset()
```

Stemming and lemmatisation

Words take different inflections in different contexts; e.g., 'wonder', 'wonderful' and 'wondering' are inflections of the word 'wonder'. Words can also take derivational forms like 'diplomacy', 'diplomatic', etc. Stemming is the process of reducing inflected or sometimes derived words to their root form, which can be used to generate words by suffix concatenation. The stemming algorithm works by removing common prefixes and suffixes from the given input. NLTK provides interfaces for the Porter stemmer, Snowball stemmer, Lancaster stemmer, etc. The following statements illustrate the use of the Porter stemmer:

```
>>> from nltk.stem.porter import *
>>> stemmer = PorterStemmer()
>>> stemmer.stem('wondering')
u'wonder'
```

Lemmatisation is the process of converting input into its dictionary form called lemma. It involves morphological analysis of the input. NLTK's lemmatiser works on the basis of WordNet, a database of English. The use of the WordNet lemmatiser is shown below:

```
>>> from nltk.stem import WordNetLemmatizer
>>> lemmatizer = WordNetLemmatizer()
>>> lemmatizer.lemmatize('wolves')
u'wolf'
>>> lemmatizer.lemmatize('is')
'is'
>>> lemmatizer.lemmatize('are')
'are'
```

We can see in the above example that 'is' and 'are' are not lemmatised to their base form 'be'. This is because the default POS argument of *lemmatize()* is 'n' ('n' stands for noun). To change the output, specify the POS argument 'v'.

```
>>> lemmatizer.lemmatize('are',pos='v')
u'be'
>>> lemmatizer.lemmatize('is',pos='v')
u'be'
```

We can see that result after stemming and lemmatisation are in Unicode format, which is indicated by 'u'.

Other data processing tasks

Apart from tasks like tokenisation, lemmatisation, etc, NLTK can also be used in tasks like finding words that are longer than a specified value, the frequency of characters, searching text, sorting the contents of a book, etc. Some of these tasks are explained below.

Frequency of occurrences: The *FreqDist()* function accepts iterable tokens. Since character-by-character iteration is possible in strings, the application of this function on strings yields the frequency distribution of all characters in that string. An example is shown below:

```
>>>a='hi this is sharon'
>>> nltk.FreqDist(a)
FreqDist({' ': 3, 'i': 3, 'h': 3, 's': 3, 'a': 1, 'o': 1,
'n': 1, 'r': 1, 't': 1})
```

You can see that the result is a dictionary. The frequency distribution of a particular character can be easily retrieved from this as shown below:

```
>>> nltk.FreqDist(a)['s']
3
```

If you wish to find the frequency of each word in a string, tokenise the string first and then apply *FreqDist()*.

```
>>> nltk.FreqDist(nltk.word_tokenize(a))
FreqDist({'this': 1, 'sharon': 1, 'is': 1, 'hi': 1})
```

Searching text: The *concordance()* function provided by NLTK searches for a keyword and returns phrases containing it. Users have the facility to set the length of the phrase and the number of phrases to be displayed at a time. The command given below will search for 'earth' in *text3* which is in the NLTK text book corpus:

```
>>> text3.concordance('earth',40,5)
Displaying 5 of 112 matches:
   earth . And the earth
   earth . And the earth was without form
   led the dry land Earth ; and the gather
   d said , Let the earth bring forth gras
   was so . And the earth brought forth gr
```

We can see that five phrases of length 40 characters are displayed on the screen.